



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application	
Course	Programming from First Principles	
Topic Name	Bags, lists, and tuples	
Topic Code	-	
Unit/Module No. & Name	2	Standard Data Types
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4-5
2.0	Meaning And Definition	6-9
3.0	Nature Or Type	10-11
4.0	Examples And Case Studies	12-14
5.0	Keywords And Touch Vocabulary	15



Savitribai Phule Pune University
Centre For Distance and Online Education

OBJECTIVE

1. This paper explains three common ways to group data in beginner programming, which are bags, lists, and tuples.
2. It uses simple English but keeps an academic tone that is suitable for undergraduate study.
3. The goal is to help students understand what each structure is, how it is defined, and when it should be used.
4. The paper also shows how these structures support careful thinking in coding tasks such as storing records, counting items, and moving through a sequence.
5. The attached audio lesson could not be transcribed in this chat environment, so the discussion follows the topic suggested by the file title and standard course content on these structures.
6. The explanation still aims to match a typical practice session where students first learn the meaning, then compare the types, and then apply them in different cases.
7. After reading, students should be able to define a bag, a list, and a tuple in their own words.
8. Students should also be able to compare them using three key features, which are order, repetition, and change over time.
9. In addition, students should be able to choose the best structure for a task and explain that choice with clear reasons.
10. Students should be able to describe at least two operations that are suitable for each structure.
11. Finally, students should be able to read short examples and identify which structure is being used, what rule it follows, and what kind of errors it helps prevent.

1.0 INTRODUCTION

1.1 Why Collections Matter

In programming, a value may be a number, a word, or a true or false flag. Many real tasks, however, need many values at once. A teacher stores many marks, a shop stores many product names, and a lab stores many measurements. If a student uses one variable for each value, the program becomes long and hard to update. Collections solve this problem by grouping values. A collection is treated as one object, but it contains many items. This supports patterns such as looping, counting, sorting, and filtering. It also supports testing because the student can test the same rule on many items, not just one.

Collections also support accuracy and fairness in handling data. When values are grouped, it is easier to apply the same check to every item. For example, a student can check every score for being in a valid range, or remove every empty word from a dataset. In academic work, this matters because the goal is not only to “make it run,” but to make it correct, clear, and easy to explain. A clear data structure supports a clear report because the reader can see what the program is storing and why.

1.2 Why “Bag,” “List,” and “Tuple” Appear in Early Courses

Bags, lists, and tuples appear early because they teach three big ideas. First, they teach how to group values. Second, they teach how rules shape meaning. Third, they teach how design choices affect later work. In many courses, a bag is discussed as an idea: a group of items where order does not matter, but repeats may matter. Some texts call this a multiset, because it is like a set with counts. A list is an ordered sequence where position matters. A tuple is an ordered group that is often fixed and not meant to change. Each one supports a different kind of thinking, so learning them helps students build better models.

These structures connect to real study tasks. A bag fits counting and frequency, which appears in text analysis, surveys, and simple statistics. A list fits steps, time order, and ranking, which appears in lab methods and result tables. A tuple fits records, which appear in almost every dataset, such as a student record, a coordinate pair, or a product record. Learning these structures helps students build models that match real meaning instead of forcing the data into an unclear shape.

1.3 Academic Tone and Simple Language

Academic writing in computing should be clear, careful, and direct. It should explain terms before using them and should connect ideas to examples. At the same time, the language here stays simple so students can focus on the concepts instead of difficult wording. The paper uses short sentences when possible. Key terms are repeated in similar ways so the meaning stays

steady. This style supports learning and also fits an AP approach that values clear definitions, consistent terms, and well chosen examples.



2.0 MEANING AND DEFINITION

2.1 Bag

2.1.1 Meaning

A bag is a collection where order does not matter. If you look at the bag, you care about what items are inside, not where each item sits. In many academic texts, a bag also allows repeated items. For example, a bag of letters from a word may contain the same letter many times, and those repeats are meaningful. A bag is useful when the task is about counting or grouping, not about sequence. If you shuffle the items in a bag, the bag is still the same because it represents a collection of items with counts, not a path with steps.

2.1.2 Definition

A bag can be defined as a multiset: a collection that stores items with possible repetition, where order is not important. The bag supports operations such as adding an item, removing an item, and counting how many times an item appears. Some programming libraries provide a direct bag type. In Python, a similar idea can be modeled using a dictionary or a Counter structure, where each unique item maps to a count. Even when a language does not offer a bag as a built in type, the concept is still important because it teaches students to focus on meaning: sometimes you only need “what and how many,” not “what and where.”

2.1.3 Key Properties

A bag is unordered, so position has no meaning. A bag supports repetition, so it can represent frequency. A bag is often mutable in practice, meaning the program can add and remove items or change counts. Because order is not tracked, many bag tasks focus on membership and counting. For example, a program can ask, “How many times did this word appear?” or “Which items appear at least once?” A bag also supports merging, where two bags are combined by adding counts for matching items. This is helpful when combining results from two groups, such as two classes that answered the same survey.

2.2 List

2.2.1 Meaning

A list is an ordered sequence of items. Order matters, so the first item is different from the second item. Lists match many real sequences, such as steps in a method, days in a week, and ranked results in a competition. In a list, duplicates are allowed, so the same value can appear many times. This matters when the goal is to keep all events, not only unique ones. In most beginner languages, a list is also mutable, so items can be changed, inserted, or removed as the program runs.

2.2.2 Definition

A list can be defined as a collection type that stores items in order and supports access by index. An index is a position number, and many languages begin indexing at zero. Common list operations include reading an item at a position, changing an item at a position, adding items, removing items, and slicing a part of the list. In Python, a list literal looks like [3, 5, 8]. A list can hold numbers, strings, or even other lists. Still, in academic work it is often best to store similar types together, such as all numbers, because that makes analysis easier and reduces confusion.

2.2.3 Key Properties

Lists preserve order, so they support sorting and ranking. Lists allow duplicates, so they support repeated events and frequency. Lists are typically mutable, which supports changing data, but also requires care because shared lists can change in more than one place. Lists support iteration, which means a loop can visit each item in order. Lists can also be nested to represent tables, such as a list of rows where each row is a list of values. This is common in simple data analysis tasks, like storing quiz scores for many students, where each inner list holds one student's scores.

2.3 Tuple

2.3.1 Meaning

A tuple is an ordered group of values that belongs together. Tuples often represent a record, such as (name, age) or (x, y). The key idea is that each position has meaning, like “first is name” and “second is age.” Many languages treat tuples as immutable, meaning the tuple cannot be changed in place. This supports safety because the record stays stable after it is created. Even when a language allows tuple like structures to be changed, tuples are still used to signal that the group should be treated as one unit with fixed fields.

2.3.2 Definition

A tuple can be defined as an ordered sequence of values, often immutable, that can store different types together. In Python, a tuple is commonly written with parentheses, such as (2, 3) or ("Riya", 19, True). Operations include indexing, iterating, and unpacking. Unpacking means assigning each tuple value to its own variable, such as x, y = (2, 3). Tuples are also used to return multiple results from a function, such as (min_value, max_value) after scanning a dataset.

2.3.3 Key Properties

Tuples keep order, so positions are meaningful. Tuples often have a fixed length, which supports stable records. Tuples are commonly immutable, which reduces accidental change and helps sharing because two parts of a program can hold the same tuple without fear that one part will modify it. Tuples can be nested and can appear inside lists. For example, a list of tuples can store many student records. Tuples can also act as keys in some structures when all their items are also immutable, which helps with tasks like grouping records by a fixed identifier.

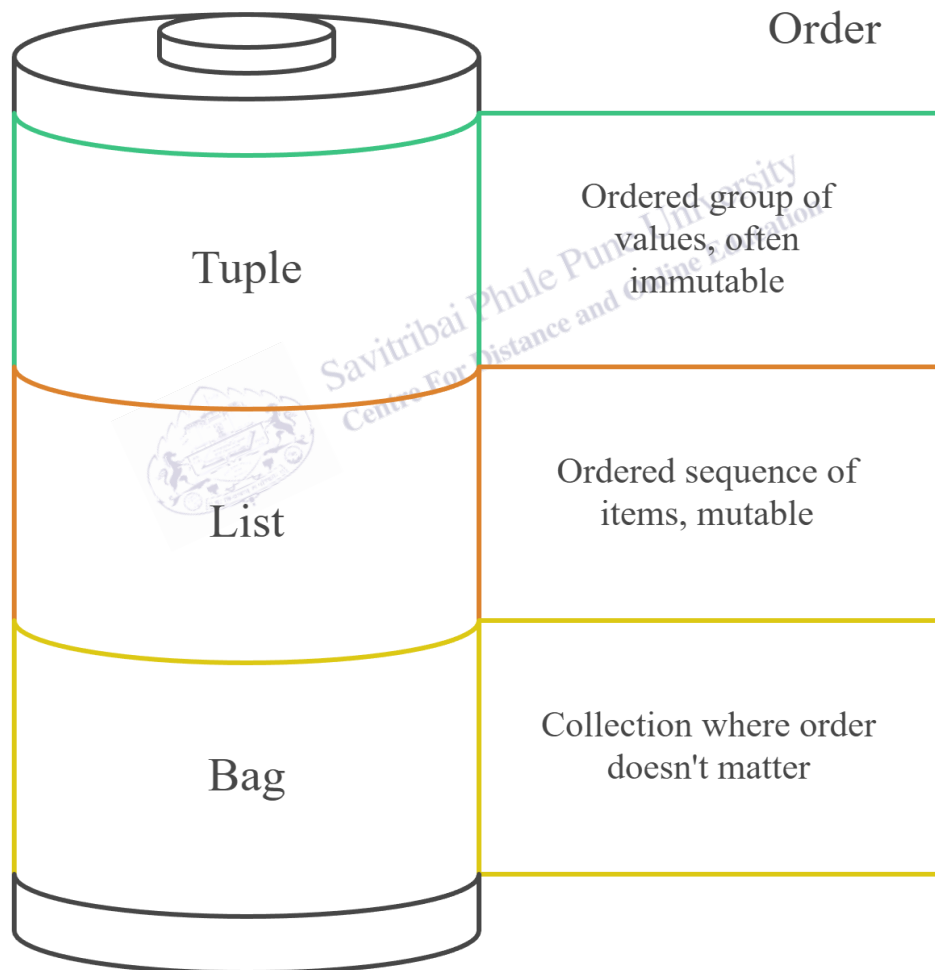
2.4 Comparing Bags, Lists, and Tuples

2.4.1 Order

Order is the clearest difference. A bag does not track order, so “red, blue” is the same as “blue, red.” A list tracks order, so [red, blue] is different from [blue, red]. A tuple also tracks order, so (red, blue) is different from (blue, red). When order reflects real meaning, such as time or rank, lists and tuples are natural choices. When order is not meaningful, a bag view can make the goal clearer and can reduce the need to manage positions.

Fig. 1

Data structures ranked by order and mutability.



The image ranks data structures by order and mutability. Tuples are ordered and usually immutable. Lists are ordered and mutable. Bags ignore order and focus on collecting values. The visual stack helps learners compare structure, behavior, and appropriate use in programming contexts.

2.4.2 Repetition

Repetition means whether duplicates are allowed and meaningful. Bags typically allow repetition because the count is part of the meaning. Lists allow repetition because repeated events may matter. Tuples can also repeat values, but repetition is less common because tuples often represent fields, not a free growing collection. Still, tuples can repeat values when a record requires it, such as (0, 0, 0) for an RGB color with equal parts, or (True, True, True) to store a fixed set of flags.

2.4.3 Change Over Time

Change over time is tied to mutability. Bags and lists are often used in situations where data grows, shrinks, or updates. Tuples are often used when data should stay fixed, such as a coordinate or an identity record. This is not only a technical rule; it is also a modeling rule. Using a tuple tells readers, “Do not change this record casually.” Using a list tells readers, “This sequence may be edited.” Using a bag tells readers, “Counts may change as we add or remove items.”



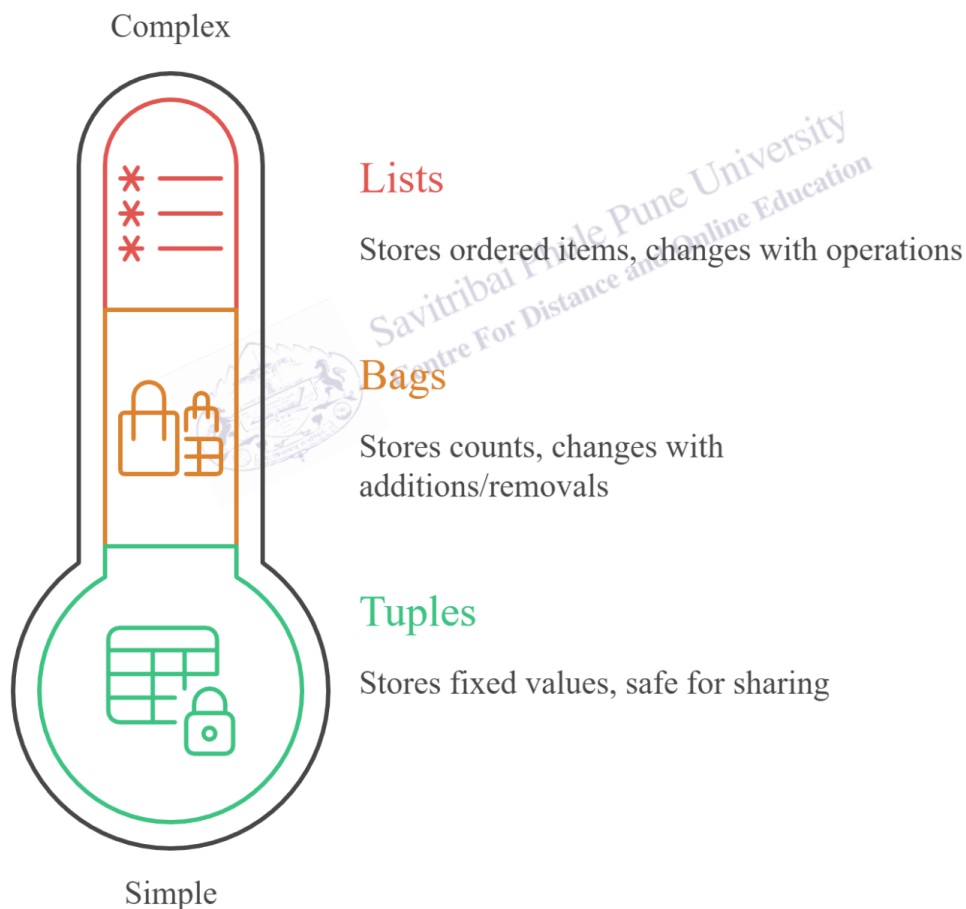
3.0 NATURE OR TYPE (IF APPLICABLE)

3.1 Primitive and Composite Types

Numbers and Booleans are often called primitive types because they store one value unit. Bags, lists, and tuples are composite types because they store many values inside one object. Composite types allow richer models, such as a list of measurements or a tuple that stores a person's details. Understanding composite types helps students move from simple code to programs that can handle real datasets, because real datasets almost always involve groups, not single values.

Fig. 2

Data structures range from simple to complex, impacting mutability and operations.



The image shows data structures from simple to complex. Tuples are simple and fixed. Bags store counts and change with additions. Lists are more complex, store ordered items, and change with operations. The thermometer design helps compare mutability and usage.

3.2 Mutability and Safety

Mutability affects safety and sharing. If a list is shared between two variables, a change through one name changes what the other name sees. This can be helpful, but it can also cause surprise during debugging. Tuples avoid this risk when they are immutable, because the values cannot be changed in place. Bags, when modeled as counters, are usually mutable because counts change as items are added and removed. In academic design, students should state when a structure is expected to change. For example, a list of weekly scores should change, but a tuple that stores a student's roll number and name should not change without a clear reason.

3.3 Memory, References, and Copies

In many languages, collections are stored as objects, and variables hold references to those objects. When you assign one list to another name, you may copy the reference, not the full list. This means both names point to the same list object. If you need an independent copy, you must create one, such as by slicing or using a copy method. Tuples, because they do not change, are simpler to share, but they can still contain mutable items, so students should be careful with nested structures. Bags, when implemented as dictionaries or counters, follow reference rules like other objects, so copying may also be needed to avoid shared updates.

3.4 Common Operations Across the Three Structures

All three structures support basic operations, but the focus differs. Bags focus on counting, such as how many times an item appears. Lists focus on positions, such as the item at index five. Tuples focus on fields, such as the second value being "age." Membership checks are common for lists and bags, and they answer the question "Is this item present?" In simple learning tasks, clarity is often more important than speed. Still, students should learn that a good structure can reduce work. For example, when counting, a bag avoids scanning the same list again and again because it stores counts directly. When modeling a record, a tuple avoids confusion because each field stays in the same place.

4.0 EXAMPLES AND CASE STUDIES

4.1 Example: Counting Words in a Short Text

Suppose a student studies a paragraph and wants to know how many times each word appears. The order of words across the paragraph may not matter for the final counts. What matters is frequency. A bag model fits well. The program reads each word, normalizes it by making it lowercase, and then updates a count. If the word is new, the count begins at one. If the word is already present, the count increases. At the end, the program can report the most common words. In an academic report, the student can explain that a bag model matches the research goal because the output is about counts, not about the original sentence order.

4.2 Example: Steps in a Lab Procedure

A lab procedure is a sequence. Step one must happen before step two. A list models this well because order is part of the meaning. A program can store each step as a short sentence in a list, such as ["Collect sample", "Add reagent", "Heat for five minutes"]. The program can display steps in order, insert a new safety step, or remove a step that is no longer needed. Because lists are mutable, the procedure can be updated as the lab plan changes. In academic writing, the student can say that a list was chosen because position carries meaning and because the method may be revised during planning.

4.3 Case Study: Coordinate Records in a Map Task

In a map task, a coordinate has a fixed number of parts. A point on a grid has two values: x and y. The pair belongs together. A tuple is a strong fit because it signals a fixed record. A list of such tuples can represent a route, where the list keeps the travel order and each tuple keeps one stable point. For example, route = [(0, 0), (1, 2), (3, 3)]. The list can grow as the route expands, but each coordinate tuple stays stable. This design is easy to read and matches the real meaning of a path made of points. It also supports analysis such as measuring distance between each pair of points in order.

4.4 Case Study: Shopping Items with Quantities

A shopping cart may contain repeated items. If the goal is to track quantities, a bag model is helpful. Each product code maps to a count. For example, “pencil” may have count three and “notebook” may have count one. A list can still be used for display, such as showing items in the order added, but for totals and inventory checks, counts are the core need. A tuple can represent a fixed product record, such as (code, name, price). Then the bag stores how many of each code the cart holds. This case shows how different structures can work together: the tuple keeps fields stable, the bag stores quantities, and a list can store the sequence of user actions if the design needs it.

4.5 Example: Student Records in a Small Class

A class roster can be modeled as a list of tuples. Each tuple stores a student record, such as (name, roll_number, program). The list keeps many students together and supports adding or removing a student. The tuple keeps the fields together and reduces accidental mixing of fields. If the program needs to count how many students are in each program, it can build a bag of program names, where each program name maps to a count. This approach supports a clear academic report: the list stores raw records, and the bag stores summary counts. The student can also explain that tuples protect field meaning because the roll number stays linked with the correct name.

4.6 Case Study: Comparing Two Collections of Items

Sometimes students need to compare two groups. For example, two study groups may list the books they read. If the goal is to compare which books appear and how often, a bag view can be useful because it tracks counts. If the goal is to compare exact reading order, a list view is needed. If the goal is to store each person's identity record, a tuple fits. This case shows that the same real situation can be modeled in different ways, depending on the research question. In an academic report, the student should state the question first, then explain why the chosen structure matches the question. This is a key habit in computing research and applied analysis.

A helpful classroom exercise is to take one real dataset and model it three ways. First, keep the raw observations in a list so you can see the time order and repeat values. Second, build a bag of the same observations so you can see how often each value occurs without caring about position. Third, store a few key facts as tuples, such as (topic, date, source), to keep records stable while you analyze. When students compare results, they learn that no structure is best for every goal. The best choice depends on the question you are answering and on what kind of mistakes you want to prevent. This habit supports academic honesty because you can explain your method and show why your structure matches your claim. In reports, students can note that lists support sequence, bags support frequency, and tuples support stable fields, then cite examples clearly from their own data.

4.7 Example: Packing Items for a Trip

A student planning a trip may track items to pack. If the student wants a checklist in a chosen order, a list fits because the student may prefer to pack clothes first and tools later. If the student also wants to know counts, such as how many socks to pack, a bag model can help by mapping “socks” to a number. If the student stores fixed details about the trip, such as (city, start_date, end_date), a tuple fits because these fields belong together and should not shift by accident. This example shows a simple but realistic modeling choice: use a list for sequence, a bag for quantity, and a tuple for stable details.

1. Bags support frequency, so they fit counting tasks and quantity tracking.

2. Lists support order, so they fit steps, time series, and ranked outputs.
3. Tuples support fixed records, so they fit grouped fields like coordinates and IDs.
4. Choose the structure that matches meaning, not only what runs without errors.
5. Use clear names so readers know what each collection stores and why it exists.
6. Test with small data first, then scale to larger data after the model is correct.
7. Remember that mutable structures can change through shared references.
8. Explain your choices in reports using the words order, repetition, and mutability.



5.0 Keyword Touch Vocabulary Meaning

Keyword	Touch vocabulary meaning	Use in this paper
Bag	Unordered collection with counts	Used for frequency and quantity
Multiset	Bag with repeats allowed	Academic name for bag idea
List	Ordered collection	Used for sequences and ranking
Tuple	Fixed ordered record	Used for stable grouped fields
Order	Position meaning	Key difference among structures
Duplicate	Repeated item	Allowed in bags and lists
Mutable	Can change	Common for lists and bag counters
Immutable	Cannot change	Common for tuples
Index	Position number	Used to access list items
Unpacking	Split tuple into variables	Used to read tuple fields
Membership	“Is it inside?” test	Used for searching collections
Frequency	How often it appears	Central in bag models